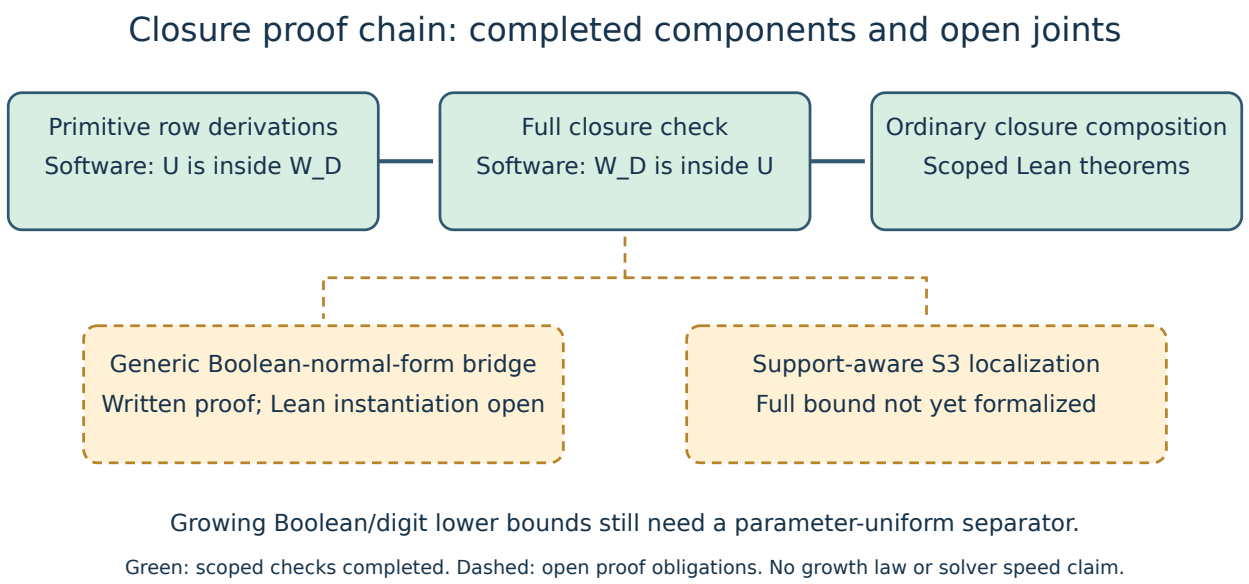
Boolean/digit last-fall proof work

4 November 2026. First implementation and proof tranche for review items 1–4.



Concrete progress

A finite certificate checker now requires both primal derivations and a closed candidate space. It rejects unproved seed-kernel rows, verifies the entire low-degree intersection after elimination, and supports prime as well as binary coefficients. Explicit polynomial repair identities account for Boolean reductions using the bit equations.

Nine deterministic software regressions pass. They include the retained eight-variable Semaev draw with full closure ranks 1, 11, 79; a closed superspace lacking primal provenance; an unclosed span whose literal inputs are all quadratic; malformed references; prime-field UNSAT; and normalization at coefficients 2, 5 and 65537. These are software controls, not new research samples.

A scoped Lean 4.35.0 module builds on the unchanged pinned Mathlib revision. Its transitive audit covers nine new theorems with no nonstandard axioms or compiler-trusting evaluation. The statements concern ordinary polynomial closure certificates, exact last fall from adjacent bounds, a SAT missing-consequence criterion, assignment-indicator partition and final localization assembly.

Exact scope

The generic Boolean-normal-form bridge is a written proof with explicit normalized-input hypotheses and raw-input degree limits. The new Lean module does not formalize that bridge, the checker, or the full binary 3 ell + 1 bound. The localization theorem requires derivability of localized rows as an explicit hypothesis. It is not renamed as the completed bound.

Acceptance conditions still open

1. Instantiate Boolean normalization and the production encoding maps in Lean. Audit each experimental generator list; equality of solution sets alone is insufficient.
2. Integrate primitive derivation provenance into large certificate producers. A dual kernel certificate establishes a closed superspace, not exact closure equality.
3. Prove support-aware localization at the intended degree cap and generic binary descent, then compose the full binary upper bound. Formalize the coefficient-field classification separately.
4. Construct separators for arbitrarily growing parameters, or a degree-controlled reduction proving lower bounds. No parameter-uniform separator has been constructed here; no growth law is proved.

Lower-bound proof route

For SAT inputs, evaluation on the entire Boolean solution set gives a linear map on degree-bounded polynomials. Its kernel is the low-degree part of the vanishing ideal. A kernel polynomial outside the checked closure is a concrete missing consequence and gives a last-fall lower bound. The checker returns such a witness when its exact evaluation-rank criterion applies. Merely failing to derive one on SAT inputs is never counted as evidence.

For UNSAT, an upper derivation of one and a lower closed separator at the preceding degree establish an exact value. The upper derivation proves unit-ideal membership, so the Lean exactness lemma does not assume a unit ideal circularly.

Sources, reproducibility and unresolved work

The source arguments and hypotheses are in THEORY.md. README.md gives commands, software limits and the status of all four tracks. proof-map.json binds the exact sources and toolchain; validation/ retains failed attempts and successful checks. Independent review is tracked separately and is not implied by the author’s build or tests.

Primary context: Huang–Kosters–Yang–Yeo, arXiv:1505.02532; Caminata–Gorla, arXiv:2112.05579v2. Internal evidence: the binary m=2 dossier, its 2026-10-19 dimension correction, the frozen prime digit closure convention, the 2026-10-26 Semaev closure audit, and the current degree-five quotient-lift/primal-witness work. No literature-priority claim is made.

No scientific sweep, performance benchmark, cloud job or official research-state transition ran. Existing historical records and canonical frontier graphs remain unchanged. A passing scoped module build does not complete the generic Semaev bound or the growing-family lower-bound problem.